MDTS4214_737_5

SULAGNA ROY

2026-02-25

Problem Set 3 : MULTIPLE LINEAR REGRESSION

Sum 3 : Problem to demonstrate the role of qualitative (ordinal) predictors in addition to quantitative predictors in multiple linear regression

Consider "diamonds" data set in R.

It is in the ggplot2 package.

Make a list of all the ordinal categorical variables.

Identify the response.

```
# Calling the required package-
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.5.2
```

```
# Storing the Data Set-
df=diamonds

# Knowing the data set-
#??diamonds
```

<u>Comment:</u>

(I) The list of all the ordinal predictors-

(i)cut:quality of the cut (Fair, Good, Very Good, Premium, Ideal)

(ii)color:diamond color (from D (best) to J (worst))

(iii)clarity: a measurement of how clear the diamond is.

(I1 (worst), SI2, SI1, VS2, VS1, VVS2, VVS1, IF (best))

(II) Identification of Response:

price:price in US dollars ($326–$18,823)

(a) Run a linear regression of the response on the quality of cut.

Write the fitted regression model.

```r
#Attaching the data set-
attach(df)

df1=data.frame(cut,price)

#Number of observations
n=53940

#Matrix of Ordinal Variables-
x=matrix(0,ncol=4,nrow=n)
for(i in 1:n)
{
  if(df1[i,1]=="Ideal")
  {
    x[i,]=c(0,0,0,0)
  }else
  {
    if(df1[i,1]=="Premium")
    {
      x[i,]=c(1,0,0,0)
    }else
    {
      if(df1[i,1]=="Very Good")
      {
        x[i,]=c(1,1,0,0)
      }else if(df1[i,1]=="Good")
      {
        x[i,]=c(1,1,1,0)
      }else
      {
        x[i,]=c(1,1,1,1)
      }
    }
  }
}
colnames(x)=c("Premium","Very_Good","Good","Fair")
df2=data.frame(cut,price,x)
model=lm(price~Premium+Very_Good+Good+Fair,data=df2);model
```

```
##
## Call:
## lm(formula = price ~ Premium + Very_Good + Good + Fair, data = df2)
##
## Coefficients:
## (Intercept)      Premium    Very_Good         Good         Fair
##      3457.5       1126.7       -602.5        -52.9        429.9
```

```r
summary(model)
```

```
##
## Call:
## lm(formula = price ~ Premium + Very_Good + Good + Fair, data = df2)
##
## Residuals:
##    Min     1Q Median     3Q    Max
##  -4258  -2741  -1494   1360  15348
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  3457.54      27.00 128.051  < 2e-16 ***
## Premium      1126.72      43.22  26.067  < 2e-16 ***
## Very_Good    -602.50      49.39 -12.198  < 2e-16 ***
## Good          -52.90      67.10  -0.788  0.43055
## Fair          429.89     113.85   3.776  0.00016 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3964 on 53935 degrees of freedom
## Multiple R-squared:  0.01286,    Adjusted R-squared:  0.01279
## F-statistic: 175.7 on 4 and 53935 DF,  p-value: < 2.2e-16
```

```r
cut_type=c("Ideal","Premium","Very_Good","Good","Fair")
P=c(0,1,1,1,1)
V=c(0,0,1,1,1)
G=c(0,0,0,1,1)
F=c(0,0,0,0,1)
df3=data.frame(cut_type,P,V,G,F)
```

Comment:

The fitted regression model is:

price=beta_0+P*beta_P+V*beta_V+G*beta_G+F*beta_F+(random_error); where

definition of the variables P,V,G,F are coded as-

```r
df3
```

```
##    cut_type P V G F
## 1     Ideal 0 0 0 0
## 2   Premium 1 0 0 0
## 3 Very_Good 1 1 0 0
## 4      Good 1 1 1 0
## 5      Fair 1 1 1 1
```

Comment:

The estimated regression model is:

estimated_price=3457.5+(1126.7*P)-(602.5*V)-(52.9*G)+(429.9*F)

(b) Test whether the expected price of diamond with premium cut is significantly different from that of the ideal cut.

Hypothesis:

Here, we are to test:

H0:beta_P=0 against H1:beta_P not equal to zero,

where,

beta_P measures the average increase in the price of diamond

if the cut type changes from Ideal cut to Premium cut.

Test Statistic:

t=estimated_beta_P/se(estimated_beta_P)

Under H0,t~t(n-p-1)

where, p:No. of predictors (Here, p=4)

Critical Region:

Reject H0,

if t(obs)>t(0.025,n-p-1);

at 0.05 level of significance.

```r
summary(model)
```

```
##
## Call:
## lm(formula = price ~ Premium + Very_Good + Good + Fair, data = df2)
##
## Residuals:
##    Min     1Q Median     3Q    Max
##  -4258  -2741  -1494   1360  15348
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  3457.54      27.00 128.051  < 2e-16 ***
## Premium      1126.72      43.22  26.067  < 2e-16 ***
## Very_Good    -602.50      49.39 -12.198  < 2e-16 ***
## Good          -52.90      67.10  -0.788  0.43055
## Fair          429.89     113.85   3.776  0.00016 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3964 on 53935 degrees of freedom
## Multiple R-squared:  0.01286,    Adjusted R-squared:  0.01279
## F-statistic: 175.7 on 4 and 53935 DF,  p-value: < 2.2e-16
```

```r
#Observed Test Statistic:
t=26.067
```

```r
#Critical point:
p=4
n-p-1
```

```
## [1] 53935
```

```r
qt(0.025,df=n-p-1,lower.tail=F)
```

```
## [1] 1.960008
```

Decision Rule:

We reject H0:beta_P=0 since here, t(obs)(=26.067)>(1.960008=)t(-0.025,n-p-1=53935).

Conclusion:

In the light of the given data, we say, the expected price of diamond with premium cut is significantly different from that of the ideal cut.

(c) What is the expected price of a diamond of ideal cut?

In order to obtain the expected price of a diamond of ideal cut,

we compute P=V=G=F=0 in the estimated regression model-

estimated_price=3457.5+(1126.7*P)-(602.5*V)-(52.9*G)+(429.9*F)

Ans:3457.5 $

(d) Modify the regression model in (a) by incorporating the predictor "table". Write the fitted regression model.

```r
df4=data.frame(df3,table)
model2=lm(price~P+V+G+F+table,data=df4);model2
```

```
##
## Call:
## lm(formula = price ~ P + V + G + F + table, data = df4)
##
## Coefficients:
## (Intercept)            P            V            G            F        table
##  -9108.6184      -0.7405      -3.4460       6.0682      -1.9353     226.9871
```

Comment:

The fitted regression model is:

price=beta_01+P*beta_P1+V*beta_V1+G*beta_G1+F*beta_F1+beta_t*table+(random_error)

after incorporating the 'table' variable. The estimated regression model is:

price=-9108.6184+P*(-0.7405)+V*(-3.4460)+G*(6.0682)+F*(-1.9353)+(226.9871)*table

(e)Test for the significance of "table" in predicting the price of diamond.

Hypothesis:

Here, we are to test:

H0:beta_t=0 against H1:beta_t not equal to zero,

where,

beta_t measures the average change in the price of diamond due to 1 unit change in table value holding a fixed type of cut.

Test Statistic:

t=estimated_beta_t/se(estimated_beta_t)

Under H0,t~t(n-p-1)

where, p: no. of predictors(Here, p=5).

```
summary(model2)
```

```
##
## Call:
## lm(formula = price ~ P + V + G + F + table, data = df4)
##
## Residuals:
##    Min     1Q Median     3Q    Max
##  -6524  -2752  -1490   1368  15743
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept) -9108.6184   439.7480 -20.713   <2e-16 ***
## P              -0.7405    53.8813  -0.014    0.989
## V              -3.4460    53.8815  -0.064    0.949
## G               6.0682    53.8816   0.113    0.910
## F              -1.9353    53.8814  -0.036    0.971
## table         226.9871     7.6255  29.767   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3957 on 53934 degrees of freedom
## Multiple R-squared:  0.01616,    Adjusted R-squared:  0.01607
## F-statistic: 177.2 on 5 and 53934 DF,  p-value: < 2.2e-16
```

```
t=29.767
p=5
n-p-1
```

```
## [1] 53934
```

```
qt(0.025,df=n-p-1,lower.tail=F)
```

```
## [1] 1.960008
```

Critical Region:

Reject H0,

if t(obs)>t(0.025,n-p-1);

at 0.05 level of significance.

Decision Rule:

We reject H0:beta_t=0 since here, t(obs)(=29.767)>(1.960008=)t(-0.025,n-p-1=53934).

Conclusion:

In the light of the given data, we say, the table variable is significant in predicting the price of the diamond.

Problem Set 5 : K nearest neighbour regression

Sum 1 : Problem to demonstrate the utility of K nearest neighbour regression over least squares regression

Consider a setting with n = 1000 observations.

```
rm(list=ls())
n=1000
```

Generate

(i) x1i from N(0,2^2) and x2i from Poisson(mean = 1.5).

```
set.seed(123)
x1=rnorm(n,0,2)
set.seed(123)
x2=rpois(n,1.5)
```

(ii) ei from N(0,1).

```
set.seed(123)
e=rnorm(n)
```

(iii) yi = -2 + 1.4*x1i* - *2.6*x2i + ei.

```
y=(-2) + (1.4*x1) - 2.6*x2 + e
```

Split the data into train and test sets.

Keep the first 800 observations as training data and the remaining as test data.

```
#Total data set:
df=data.frame(y,x1,x2)

#Training data set:
train=df[1:800,]

#Test data set:
test=df[801:n,]
```

Work out the following:

1.Fit a multiple linear regression equation of y on x1 and x2.

```
model1=lm(y~x1+x2,data=train);model1
```

```
##
## Call:
## lm(formula = y ~ x1 + x2, data = train)
##
## Coefficients:
## (Intercept)           x1           x2
##        -2.0          1.9         -2.6
```

Comment:

The estimated model is:

estimated_y= -2 + (1.9*x1)* - *(2.6*x2)

Calculate test MSE.

```
y_est_test=predict(model1,data=test)
test_mse=mean((test[,1]-y_est_test)^2);test_mse
```

```
## [1] 50.53066
```

Comment:

The test mse is 50.53066.

2. Fit a KNN model with k = 1,2,5,9,15.

Calculate test MSE for each choice of k.

```
k = c(1,2,5,9,15)
mse_knn=rep(0,5)
for(i in 1:5)
{
  library(FNN)
  knn_model <- knn.reg(train = train,
                       test  = test,
                       y     = train$y,
                       k     = k[i])
  predictions <- knn_model$pred
  mse_knn[i]=mean((test$y-predictions)^2)
};mse_knn
```

```
## Warning: package 'FNN' was built under R version 4.5.2
```

```
## [1] 0.02065269 0.01752089 0.03818303 0.06282994 0.09979219
```

Suppose the data in Step (iii) is generated as :

yi = 1/(-2 + 1.4*x1i* - *2.6*x2i + 2.9*(x^2)1i)* + *3.1*sin(x2i) - 1.5*x1i*(x2^2)i + ei.

Work out the problems in (1) and (2). Compare and comment on the results.

```
#New Response
y=1/(-2 + 1.4*x1 - 2.6*x2 + 2.9*(x1^2) ) + 3.1*sinpi(x2) - 1.5*x1*(x2^2) + e

#New data set
df=data.frame(y,x1,x2)

#New train data set
train=df[1:800,]

#New test data set
test=df[801:n,]
```

1.

```
model2=lm(y~x1+x2,data=train);model2
```

```
##
## Call:
## lm(formula = y ~ x1 + x2, data = train)
##
## Coefficients:
## (Intercept)           x1           x2
##      0.2349      -4.5087       0.2689
```

<u>Comment:</u>

The estimated model is:

estimated_y= 0.2349 - 4.5087*x1* - *0.2689*x2

Test MSE:

```
y_est_test=predict(model2,data=test)
test_mse=mean((test[,1]-y_est_test)^2);test_mse
```

```
## [1] 343.4743
```

<u>Comment:</u>

The test mse is 343.4743.

##2.

```r
k = c(1,2,5,9,15)
mse_knn=rep(0,5)
for(i in 1:5)
{
  library(FNN)
  knn_model <- knn.reg(train = train,
                       test  = test,
                       y     = train$y,
                       k     = k[i])
  predictions <- knn_model$pred
  mse_knn[i]=mean((test$y-predictions)^2)
};mse_knn
```

```
## [1]  0.3919643  0.2853503  0.9585152  4.0541684 10.1129010
```